

NexusAI security review

Julian Calvo. AAI6850, Module 15 capstone. August 2026. Scope: contracts/ at the state in this submission.

This is my own review of my own code, not an independent audit.

1. Summary

I reviewed the ten contracts in three passes:

1. Static analysis. Slither 0.11.6 (100 detectors) and Solhint 5.x over the whole contract set.
2. Adversarial tests. 195 tests including a hostile ERC20 that re-enters the payment path, a rogue registry trying to inject listings, a replayed bridge message, a post-snapshot governance takeover attempt, and an access-control matrix that checks every privileged entry point rejects an unprivileged caller.
3. Manual reading. Line by line through every state transition against a written invariant list, then a second pass comparing the docs to the code.

13 issues found, 12 fixed. No critical issue. The two High findings were both found by reading, not by tooling, and both broke the same thing: the provider bond, which is what the whole economic security story rests on.

Severity	Found	Fixed
Critical	0	0
High	2	2
Medium	5	5
Low	3	3
Informational	3	2
Total	13	12

Slither produced 45 results. 4 led to code changes, 41 are accepted with a reason written down. 23 of those 41 are the timestamp detector firing on time-based logic that is deliberate.

Eight residual risks are listed in section 4.

2. Method

```
npm run lint          # solhint
npm run build         # compile
npm test              # 195 tests
npm run coverage      # 99.76 / 92.56 / 99.15 / 99.82
slither . --config-file slither.config.json
```

The Slither config excludes node_modules and the OpenZeppelin dependency tree, so results are about code I wrote. Raw output is in audit/slither-report.md.

Solhint: 0 errors, 7 warnings, all style, all accepted.

3. Findings

H-01. Provider bond could be withdrawn in the same block it was posted

The marketplace checked that a provider had a bond, but not that the bond was locked past the dispute window. A provider could stake, list, sell a licence, and unstake in one transaction. The bond looked like it was there and secured nothing.

Fix: `registerModel` now requires the stake's unlock time to be at least `disputeWindow` past the licence term.

Test: "refuses a bond that is not locked long enough (stake/list/unstake attack)".

H-02. `minProviderBond` defaulted to zero

Bonding was enforced, but against a floor of zero, so any provider passed with nothing at stake. Same effect as H-01 by a different route.

Fix: constructor takes a non-zero minimum and reverts on zero. Default is now 10,000 NEXA.

M-01. Winning an exclusive auction gave you no exclusivity

`settle` paid the seller and marked the auction complete but never issued a licence, and never stopped anyone else buying the model normally. Nothing about it was actually exclusive.

Fix: added `issueExclusiveLicence` on the marketplace, gated by `LICENCE_ISSUER_ROLE` held only by the auction contract, plus a per-model exclusivity flag that blocks ordinary purchases while it's set.

M-02. A hub mirror could re-list a model a spoke had delisted

Mirroring overwrote local state unconditionally, so a delisted model came back the next time anyone republished.

Fix: `mirrorModel` no longer reactivates a locally delisted model.

M-03. Genesis mint charged against the epoch budget

The 400M genesis mint went through the same accounting as ordinary emissions, so it consumed the epoch ceiling many times over. `mint()` and `remainingEmission()` then reverted on underflow.

Fix: genesis mint happens in the constructor and bypasses epoch accounting.

M-04. Lowering the emission ceiling below current usage reverted all minting

`remainingEmission()` computed `ceiling - mintedThisEpoch` without a floor, so if governance lowered the ceiling mid-epoch the subtraction underflowed and every mint reverted until the epoch rolled.

Fix: clamp at zero.

M-05. Exclusive licence issuance skipped the compliance gate

I introduced this while fixing M-01. `issueExclusiveLicence` wrote the entitlement directly without checking `requiresCompliance`, so the auction path was a way around the compliance check that the ordinary purchase path enforces.

Found on a later manual pass, not by any tool. Fix: the same compliance check now runs on both paths. Test file `test/Exclusivity.test.js`.

L-01. ETH sent to CrossChainRegistry was stranded

The registry takes `msg.value` to pay router fees but had no way to get overpaid value back out.

Fix: added `sweep(address)` behind `BRIDGE_ADMIN_ROLE`.

L-02. The registry's marketplace address was mutable

An admin could point the registry at a different marketplace after deployment, which is a large amount of trust for no benefit.

Fix: made it immutable.

L-03. setCompliance emitted no event

Silent privileged change. Fix: added `ComplianceUpdated`.

INFO-01. Missing zero-address check on withdrawFees

Fixed.

INFO-02. require with a string instead of a custom error

Fixed, for consistency with the rest of the file.

INFO-03. Reentrancy exposure on the payment path

Not fixed. I don't think there's anything here to fix. `purchaseLicence` follows checks-effects-interactions and carries `nonReentrant`. I wrote `ReentrantToken`, a malicious ERC20 that calls back into the marketplace during `transferFrom`, to confirm it. The reentry reverts. Accepted, with the test as evidence.

4. Residual risks

These are open, known, and in the roadmap.

ID	Risk	Severity	Why it's tolerable for now
R-01	No cross-chain licence revocation. A refunded buyer keeps a mirrored licence on a spoke until it expires	Medium	Licences are 30 days by default and mirroring is a manual operator action, so an operator aware of a dispute can just not mirror
R-02	Rewards accrued while the vault is empty are stranded	Low	Only reachable before the first stake. Nobody can take them
R-03	The 5% protocol fee isn't refunded on a dispute	Low	Bounded at 5% and disclosed. Escrowing it would delay treasury income on every honest sale to insure a rare dishonest one
R-04	A majority of oracle reporters could collude	Medium	Median defeats a minority, the circuit breaker catches large moves, reporters are permissioned and identifiable
R-05	The router is a trust boundary. A compromised router can call <code>ccReceive</code> with anything	Medium	The registry checks the trusted remote and the nonce itself, so a compromised router still can't forge a sender. It could suppress or reorder
R-06	ML results are on synthetic data	Medium (for the claims, not the contracts)	Stated in the ML report and the model card. Real-corpus loaders are written and attempted first
R-07	Exclusivity doesn't propagate cross-chain	Low	Exclusive auctions are a hub-side product today
R-08	<code>releaseEscrow</code> reverts <code>NoOpenDispute</code> when a dispute is open	Informational	Behaviour is right, the name is backwards, renaming breaks the interface

5. Before mainnet

1. Get an independent audit.
2. Re-run `governance-handoff.js` on any new chain and confirm every row says PASS. It's already done on the Sepolia hub.
3. Fix R-01 before turning on automatic licence mirroring.
4. Replace `MockCrossChainRouter` with a real CCIP or LayerZero adapter and redo this whole review against it.
5. Stand up the monitoring in `docs/monitoring.md`, especially oracle staleness and the circuit breaker. The breaker is useless if nobody gets paged when it trips.
6. Formally verify the reward accumulator. R-02 is evidence the invariant doesn't hold exactly, and that's the kind of arithmetic property a prover handles better than tests do.
7. Re-run the ML on real corpora before quoting an accuracy number commercially.
8. Add Foundry invariant tests: total staked equals vault balance minus escrowed rewards, licence expiries are monotonic, the median lies inside the min and max of fresh reports.

6. Conclusion

The code enforces role boundaries, reentrancy protection, router and remote authentication, replay protection, timelocked governance, and snapshot voting. Each of those has a named test.

Static analysis found neither High finding. Both were provider-bond conditions that looked present and did nothing. M-05 was introduced while remediating another finding and caught on a later pass.

I found no path for an unprivileged party to steal funds, mint past the cap, forge an attestation, or bypass the quality gate in the code I reviewed. I cannot say there is nothing there.

This is a capstone project. It has not been independently audited and must not be deployed to mainnet or used to hold real value without the work in section 5.

Appendix: invariants the tests assert

- Total supply never exceeds 1,000,000,000 NEXA
- Minting in an epoch never exceeds `emissionCeiling`, which never exceeds `MAX_EPOCH_EMISSION`
- Protocol fee never exceeds `MAX_PROTOCOL_FEE_BPS` (10%)
- A single slash never exceeds `MAX_SLASH_BPS` (30%)
- A purchase never succeeds while the model is below its accuracy floor, stale, or circuit-broken
- A purchase never succeeds for a non-compliant buyer on a compliance-required listing
- The same bridge nonce is never executed twice
- `ccReceive` never accepts a caller other than the configured router
- Voting power is always read at the proposal snapshot, never at vote time
- Escrow released plus escrow refunded never exceeds escrow taken